# Free Question Answering Service with LLama 2 model and Prompt Template

Alex Yeo · Follow
7 min read · Oct 22, 2023

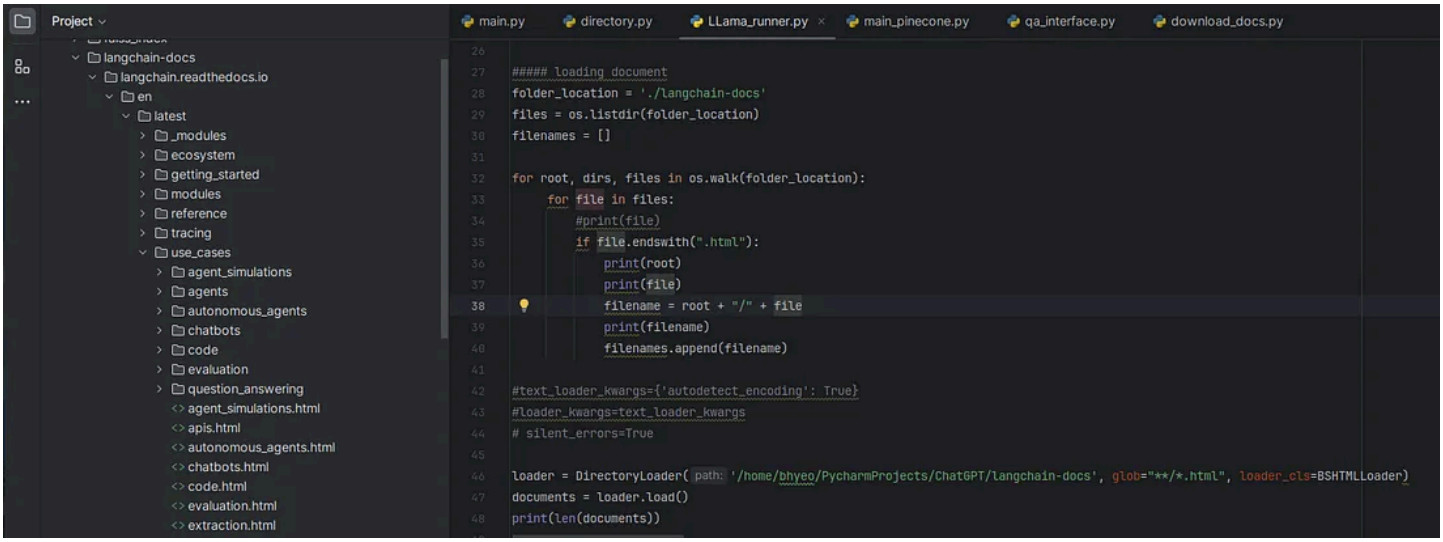


LLama is assisting a student

Content Generation (CG-AI) is the most fancy term now with the introduction of ChatGPT, LLama, DALL-E and Diffusion Model.

Some of these services require subscription fee because these applications provide the simplicity to integrate their services with your applications. If we know the steps/processes to integrate our applications with the CG-AI services, we can use the services for free.

Therefore, in this Articles, we will train LLama2 with local HTML files information and use it as our personal Question Answering bot. Even better, we can use this service for free. Yes, use it for free!




HTML files in “langchain-docs” folder

I have downloaded the langchain HTML files locally, but you can download any HTML files that you like and feed the HTML files to LLama 2. Before feeding the HTML files to LLama 2 model, we need to pre-process the HTML files and configure LLama 2 model to run the model effectively. For that to happen, we need know 3 important things : **LangChain, Transformer and HuggingFace**. LangChain contains most of the data processing functions and pipelines, Transformer contains an encoder and decoder working together for various tasks, while HuggingFace contains different types of machine learning models.

**LangChain (Data Preprocessing)**

Let’s start with LangChain. We will use few features provided by LangChain like directory loader, text splitter to pre-process our data. We can start with document loader. For this function, we need to specify the folder location and the file extensions that we are expecting. In our example, we are expecting HTML files. Then, to process these HTML files, we need a HTML parser and the parser can be found in beautiful soup library. This library is wrapped in “**BSHTMLLoader**” class file by Langchain.

```
from langchain.document_loaders import DirectoryLoader
from langchain.document_loaders import BSHTMLLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
import os

from torch import cuda, bfloat16
import transformers
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS
from langchain.llms import HuggingFacePipeline

from langchain.chains import ConversationalRetrievalChain
from langchain.prompts import PromptTemplate

model_id = 'meta-llama/Llama-2-7b-chat-hf'
device = f'cuda:{cuda.current_device()}' if cuda.is_available() else 'cpu'
```

```
hf_auth = "hf_YOUR_AUTHENTICATION_ID"
print("using : ", device)


##### loading document
folder_location = './langchain-docs'
files = os.listdir(folder_location)
loader = DirectoryLoader(folder_location, glob="**/*.html", loader_cls=BSHTMLLoader)
documents = loader.load()
print(len(documents))
```

Once we have extracted the useful contents from the HTML files, we need to split the contents into short sentences. We can achieve this by using the "**RecursiveCharacterTextSplitter**" function in LangChain. This is a simple step. Let's quickly move on to next step which is the indexing or embedding step.

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=20)
all_splits = text_splitter.split_documents(documents)
```

In the indexing step, we are converting every word in the sentences into a vector of number. For example, we could represent the word "LLama" to [0.2, 0.1. 0.3, 0.4, 0.9]. This vector of number is called embedding. For this process, we could use a pre-trained model to do the conversion. There are a lot of pre-trained models in the market, we are using FAISS model to do the indexing. If you are familiar with this process, you will know that we only need to do this once for our document. When we have new documents to feed to LLama 2 model, then, we need to do the indexing again. Therefore, I save the embedding of the documents locally so that I don't need to do indexing every time.

```
embeddings = HuggingFaceEmbeddings(model_name=model_name, model_kwargs=model_kwargs)
# storing embeddings in the vector space. do this once only.
vectorstore = FAISS.from_documents(all_splits, embeddings)
# save embedding file
vectorstore.save_local("faiss_index")
# load embedding
loaded_vectorstore = FAISS.load_local("faiss_index", embeddings)
```

#### Transformer (Data Modeling)

Great, so far so good! We have done with the data pre-processing parts. Next, we will go to the modeling part. In this part, we will configure the model and create rules for the model to behave nicely using the prompt template. Let's start with the configuration of the LLama 2 Large Language Model. The configuration that we will make here is **Quantization**. Basically, the quantization process is to reduce the model weights to 4 bits floating number. This will reduce some precision but it will increase the prediction speed drastically. This is shown in the "**# setup quantization**" part.

After that, we need to choose the task that we want to perform using the transformer model. As we know earlier, Transformer contains a encoder and decoder that can be used on various task. One of the tasks is machine Translation. if you are interested in it, you could refer to these 2 articles (Baby steps in Neural Machine Translation Part 1 (Encoder), Baby steps in Neural Machine Translation Part 2 (Decoder) ) for deeper understanding. For now, we will choose the "**AutoModelForCausalLM**" model for our Question Answering application. This part is shown in the "**# configure LLM to use quantization**" part.

```
##### setting up LLM #####
# setup quantization
```

```
bnb_config = transformers.BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=bfloat16
)

# load llm model
model_config = transformers.AutoConfig.from_pretrained(
    model_id,
    use_auth_token=hf_auth
)

# configure LLM to use quantization
model = transformers.AutoModelForCausalLM.from_pretrained(
    model_id,
    trust_remote_code=True,
    config=model_config,
    quantization_config=bnb_config,
    device_map='auto',
    use_auth_token=hf_auth
)

# enable evaluation mode to allow model inference
model.eval()
```

By now, we have configure the model properly. The next thing we want to do is to add in a tokenizer to the transformer model. So, what is the purpose of a tokenizer? if you have gone through the article "Baby steps in Neural Machine Translation Part 1" earlier, you might know the purpose of a tokenizer. Basically, a tokenizer is the way to split a sentence into a list of words. the easiest way is to use "space" as a indicator to split the sentence. There are a lot of researches have been done on the tokenizer, one of the famous tokenizer is byte-pair encoding. But this is out of the scope of this article. We will use the tokenizer suggested by the Transformer library to perform the Question Answer Task.

```
##### setup pipeline
# setup tokenizer
tokenizer = transformers.AutoTokenizer.from_pretrained(
    model_id,
    use_auth_token=hf_auth
)
# configure LLM pipeline with the tokeniser and task
generator = transformers.pipeline(
    model = model,
    tokenizer=tokenizer,
    return_full_text = True, # langchain expects full text
    task='text-generation',
    #stopping_criteria=stopping_criteria, # without this model rambles during chat
    temperature=0.1, # 'randomness' of outputs, 0.0 is the min and 1.0 is the max
    max_new_tokens=512, # max number of tokens to generate in the output
    repetition_penalty=1.1 # without this output begins repeating
)
```

Up until here, we are done with the data modeling parts. Next thing, we will proceed with data-processing part by setting the rules for the LLM Model to behave.

**HuggingFace (Data Post-Processing)**

In this part, we will wrap the Transformer model with HuggingFace pipeline so that we can pass the rules to the Transformer model. To craft and pass the rules to the Transformer model, we can use the LangChain Prompt Template. In this prompt template, we can tell how the LLM should behave. This is shown in the "pre_prompt"

variable. Next, we give some information or context to the LLM to refine our prediction. For example, we can tell the LLM that we are dicussing “Apple” product such as iphone, ipad and mac book, instead of discussing “Apple” as a fruit. With the context, we can pass in the relevant question as shown in the “prompt” variable.

Lastly, we want to chain everything together. These include, indexed HTML contents, Large Language Model, and Prompt Template(Rules). We can do this by using the HuggingFace class called “**ConversationalRetrievalChain**”.

```
e the next agent response by answering the question. Answer it as succinctly as possible. You are provided several
```

**Prediction**

Great job by following all the steps until here. Everything is ready now. We can pass in our questions to the Question Answer service. Have fun and play with it for free!

Trained Knowledgeable LLama

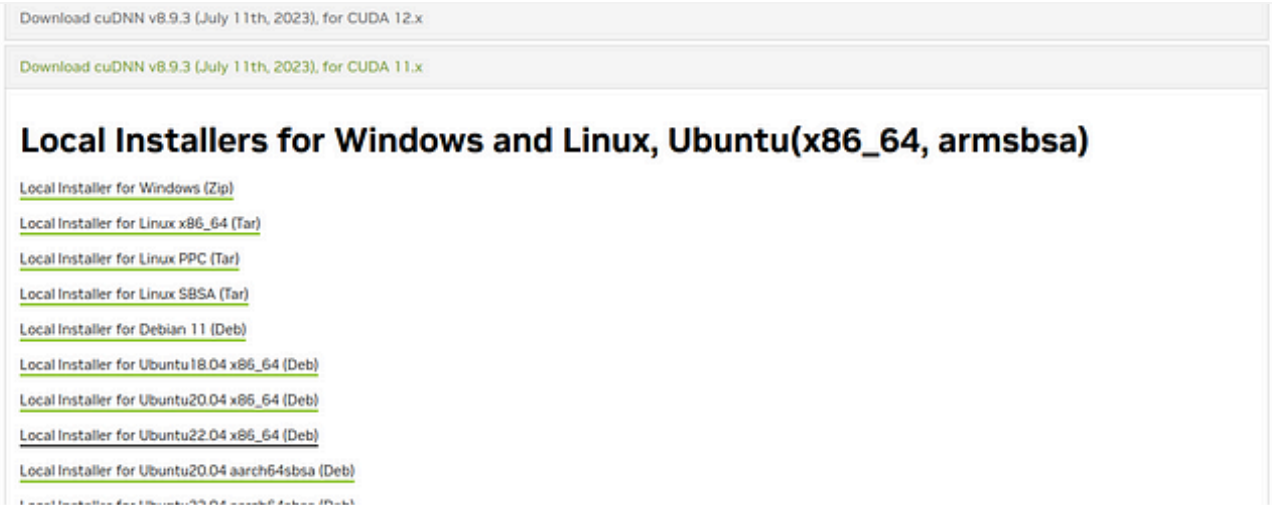
```
# testing the model

chat_history = []

query = "What is LangChain and what applications can be created using LangChain?"

result = chain({"question": query, "chat_history": chat_history})

print("answer", result['answer'])
chat_history = [(query, result["answer"])]

query = "Please repeat the applications mentioned just now?"
result = chain({"question": query, "context" : result["answer"], "chat_history": chat_history})

print("answer", result['answer'])
print("source_documents : ", result['source_documents'])
```

Hope you enjoy the article and hope this article give your extra understanding on the LLama Large Language Model.

Before leaving, I have a shameless promotion on my **Free** Udemy course : Practical Real-World SQL and Data Visualization. I find that the free visualization tool, Metabase, is very useful. Therefore, I spend weekends creating the course and I wish you could benefit from the course. Lastly, I really appreciate if you could take a look on the course and even better, please give me a review on this course so that I can improve the course. Thank you!!!

Large Language Models | Artificial Intelligence | Deep Learning | Free | ChatGPT





| | o | l |
|---|---|---|

all possible alignments

| | t3 | t4 | t5 | t6 |
|---|---|---|---|---|
| | o | - | o | l |
| | - | o | o | l |
| | o | - | o | l |

| | x | y | count | |
|---|---|---|---|---|
| t1 | wav1 | p | 3 | t4 |
| | wav1 | o | 0 | |
| | wav1 | o | 0 | |
| | wav1 | l | 0 | |
| | wav1 | - | 0 | |
| t2 | wav2 | p | 1 | t5 |
| | wav2 | o | 2 | |
| | wav2 | o | 0 | |
| | wav2 | l | 0 | |
| | wav2 | - | 0 | |
| t3 | wav3 | p | 0 | t6 |
| | wav3 | o | 2 | |
| | wav3 | o | 0 | |
| | wav3 | l | 0 | |
| | wav3 | - | 1 | |

Alex Yeo in Machine Growth

## Speech Recognition — Connectionist Temporal Classification (CTC)

Step-by-step guidance for Speech recognition loss function.

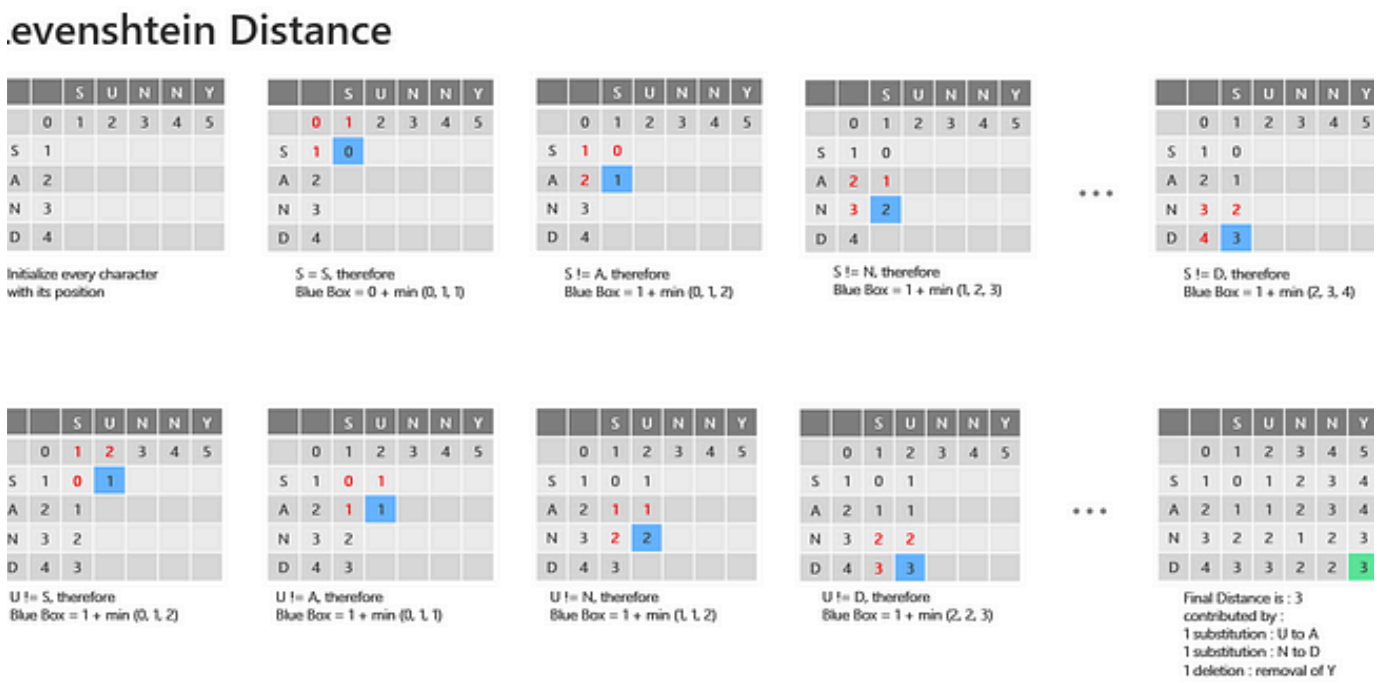
Aug 30, 2020 11

Alex Yeo

## Cosine Similarity — Introduction and applications in NLP

How similar is Father and Mother? When you ask this question to anyone, you will have different answers. Some say, mum can cook, dad can't...

Oct 4, 2020 4




Alex Yeo

## Commonly-used distance calculation in Data Science : Levenshtein Distance, Haversine Distance

There are multiple distance calculations commonly used in data science projects depending on the use cases. Usually, these distance...

Aug 1, 2023

See all from Alex Yeo

## Recommended from Medium

Gaurav

### Building Llama 3 ChatBot Part 2: Serving Llama 3 with Langchain

In the first part of this blog, we saw how to quantize the Llama 3 model using GPTQ 4-bit quantization. You can continue serving Llama 3...

Apr 29 12 2


Gayani Parameswaran

### Q&A ChatBot using Langchain, Hugging Face, and Mistral

In this blog post, we'll delve into creating a Q&A chatbot powered by Langchain, Hugging Face, and the Mistral large language model (LLM)...

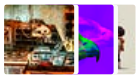
May 27 1

Lists

**ChatGPT**
21 stories · 729 saves

**AI Regulation**
6 stories · 516 saves

**ChatGPT prompts**
48 stories · 1819 saves



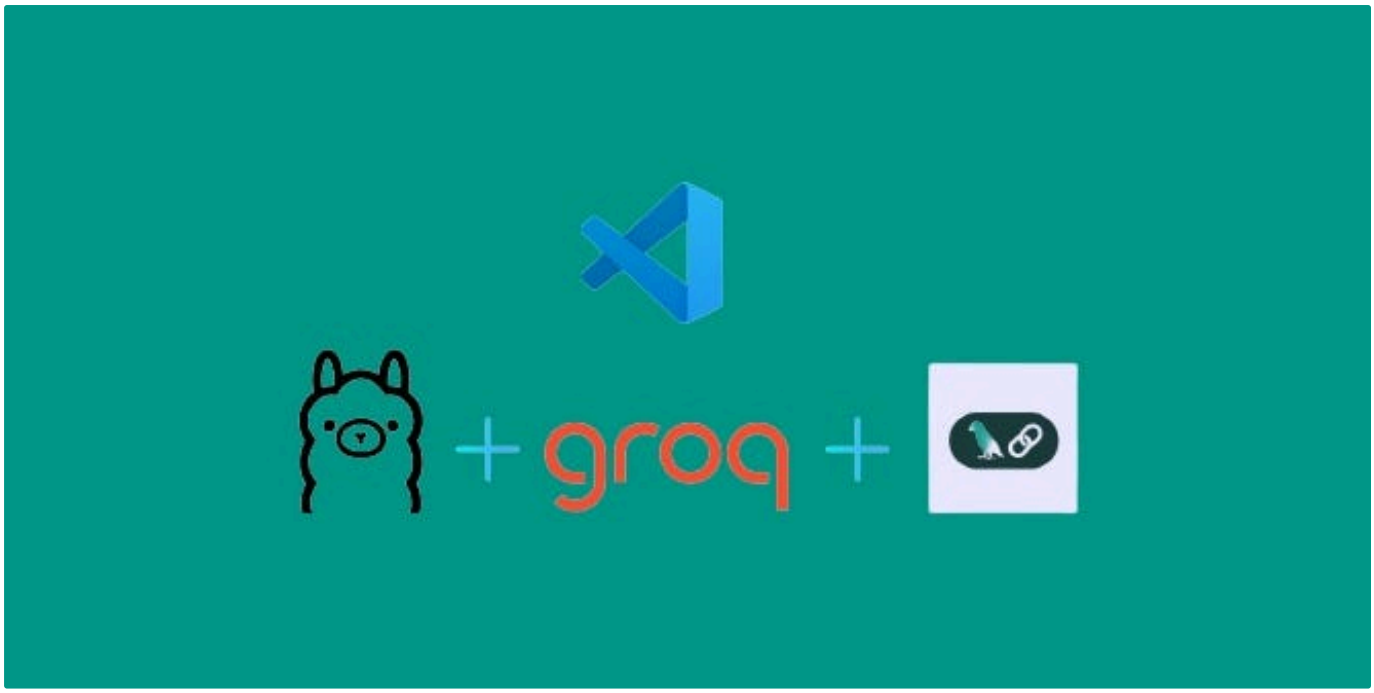
**What is ChatGPT?**
9 stories · 395 saves


Vishnu Sivan in The Pythoneers

## Unlocking Llama 3: Your Ultimate Guide to Mastering Llama 3!

The world of artificial intelligence has been rapidly evolving in recent years, largely due to the emergence of Large Language Models...

Apr 30 135 2




Raghunaathan in GoPenAI

## Implementing A Local RAG with LangChain and Llama3: A Quick Guide

Effectively Interact with Your Local Documents Using a Reliable RAG Model

May 20 10


Manuel

## Implementing and Running Llama 3 with Hugging Face's Transformers Library

Step-by-Step Guide to Run Llama 3 with Hugging Face Transformers

May 27 67 1


Gen. David L.

**Building your private knowledge base using RAG with llama3 and langchain**

In today's era of information explosion, we face the challenge of vast amounts of public or private data. Retrieving useful information...

May 20 1

See more recommendations